CS-471: Machine Learning

Week 1 — Introduction to Machine Learning & Linear Regression

Instructor: Dr. Hashir Kiani

Comprehensive Study Notes

Week 1

Contents

1 What is Machine Learning? 2
1.1 Definition . . . 2

2 Taxonomy of Machine Learning 2

3 Supervised Learning 3
3.1 Formal Definition . . . 3
3.2 Housing Price Prediction — A Running Example . . . 3
3.3 Regression vs. Classification . . . 4
3.4 More Supervised Learning Applications . . . 4
3.4.1 Image Classification . . . 4
3.4.2 Object Localisation and Detection . . . 4
3.4.3 Machine Translation . . . 5

4 Unsupervised Learning 5
4.1 Definition . . . 5
4.2 Clustering . . . 5

5 Reinforcement Learning 6
5.1 Definition . . . 6
5.2 AlphaGo — A Famous Example . . . 6

6 Real-World ML Applications 7
6.1 Machine Translation . . . 7
6.2 Autonomous Driving . . . 7
6.3 Healthcare . . . 7
6.4 Large Language Models (LLMs) . . . 7

7 ML Risks and Ethical Concerns 7

8 Linear Regression — The Simplest ML Model 8
8.1 Representing the Hypothesis h . . . 8

8.1.1 Single Feature . . . 8
8.1.2 Multiple Features . . . 8
8.2 The Three Pillars of Linear Regression . . . 9
8.3 Hypothesis Class . . . 9
8.4 Loss Function . . . 10
8.4.1 Residual . . . 10
8.4.2 Squared Loss (per example) . . . 10
8.4.3 Training Loss (over all N examples) . . . 10
8.5 Gradient of the Training Loss . . . 11
8.6 Optimisation Algorithm: Gradient Descent . . . 11
8.7 Gradient Descent on Large Datasets . . . 12
8.8 Stochastic Gradient Descent (SGD) . . . 12

9 Worked Examples **13**
9.1 Example 1 — Computing Training Loss . . . 13
9.2 Example 2 — One Step of Gradient Descent . . . 14

10 Practice Questions **14**

11 Key Takeaways **16**

1 What is Machine Learning?

1.1 Definition

Definition: Machine Learning (ML) is a field of study that gives computers the ability to learn from *data* without being explicitly programmed for every task.

Key Idea

Three core ideas:

- **Data** — the raw material; pairs of inputs and (sometimes) outputs.
- **Model** — a mathematical function whose *parameters* are found from data.
- **Prediction** — once trained, the model generalises to new, unseen inputs.

Concept Explanation (step by step):

1. You collect a dataset (e.g. house sizes and their sale prices).
2. You choose a model (e.g. a straight line passing through the data).
3. The ML algorithm *adjusts the model's parameters* so that it fits the data as closely as possible.
4. You now have a trained model that can predict the price of a *new* house it has never seen before.

Real-Life Analogy: Think of a child learning to identify cats. Nobody programs the child with exact rules ("has four legs AND whiskers AND fur…"). Instead, parents show thousands of images, labelling them "cat" or "not cat". Over time, the child's brain *learns* the right pattern. Machine Learning does exactly the same, but with maths.

2 Taxonomy of Machine Learning

The lecture presents ML as three major branches:

Branch	Core Idea	Classic Examples
Supervised Learning	Learn from labelled data (x_i, y_i) pairs.	House price prediction, spam detection, image classification
Unsupervised Learning	Find hidden structure in *unlabelled* data x_i.	Clustering customers, gene grouping, anomaly detection
Reinforcement Learning	An *agent* takes actions in an environment and learns from rewards/penalties.	Game playing (AlphaGo), robotics, autonomous driving

Real-Life Analogy for the taxonomy:

- **Supervised** ⇒ learning with a teacher who marks every answer correct or wrong.

- **Unsupervised** $\Rightarrow$ exploring a new city with no map — you discover the structure (neighbourhoods, landmarks) on your own.
- **Reinforcement** $\Rightarrow$ a dog learning tricks: it tries an action, receives a treat (reward) or scolding (penalty), and adjusts future behaviour.

3 Supervised Learning

3.1 Formal Definition

In supervised learning a **prediction (hypothesis) function** maps from input space $\mathcal{X}$ to output space $\mathcal{Y}$:

$$h : \mathcal{X} \to \mathcal{Y}$$

- $\mathcal{X}$ — the *input* (features, e.g. house size, pixel values, email text).
- $\mathcal{Y}$ — the *output* (label, e.g. price, category).

The algorithm receives a **training set** — a collection of (x_i, y_i) pairs — and its goal is to produce a hypothesis h that accurately predicts y for inputs x it has *not* seen before.

Two flavours of supervised learning:

Type	Output $\mathcal{Y}$	Example
Regression	Continuous (real number)	House price, temperature
Classification	Discrete (class label)	Spam/Not spam, cat/dog

3.2 Housing Price Prediction — A Running Example

Problem Statement: Given a dataset of n houses with their sizes in square feet and their corresponding sale prices, build a model that predicts the price of any new house given its size.

Single Feature Setup:

- Input feature: x = size of house (sq. ft.)
- Output label: y = price (in \$)
- Training set: $\{(x^{(1)}, y^{(1)}),\ (x^{(2)}, y^{(2)}),\ \ldots,\ (x^{(n)}, y^{(n)})\}$

Multi-Feature Extension: Real houses have more than one feature:

$$\mathbf{x} = [x_1,\ x_2,\ x_3,\ \ldots,\ x_n]$$

For example:

- x_1 = house size (sq. ft.)
- x_2 = number of bedrooms
- x_3 = age of house (years)
- x_4 = distance from city centre (km)

The model uses *all* features simultaneously to make a better prediction.

Key Idea

Why more features help: A house price depends on many factors, not just size. Adding relevant features gives the model more information, leading to more accurate predictions. However, adding *irrelevant* or *redundant* features can hurt performance — this is a topic revisited in later weeks (Feature Selection).

3.3 Regression vs. Classification

- **Regression:** Predicts a *continuous* output.
  - Output can take any real value.
  - Example: predicting salary, stock price, temperature.
- **Classification:** Predicts a *discrete* class label.
  - Output is one of a finite set of categories.
  - Example: email is "spam" or "not spam"; image contains a cat, dog, or bird.

Real-Life Analogy:

- Regression $\Rightarrow$ estimating how many minutes you will be late for class (any number from 0 to ∞).
- Classification $\Rightarrow$ deciding whether to take an umbrella (yes/no) — a discrete decision.

3.4 More Supervised Learning Applications

3.4.1 Image Classification

Given an image as input, assign it a class label.

- Input $\mathcal{X}$: pixel values of the image.
- Output $\mathcal{Y}$: one of many possible categories (e.g. "cat", "dog", "plane").
- Real-world use: social media photo tagging, medical imaging (classifying tumours as benign/malignant).

3.4.2 Object Localisation and Detection

A harder task than classification:

- **Localisation:** identify *where* one object is — draw a bounding box.
- **Detection:** identify *where multiple objects* are and what class each belongs to.
- Real-world use: self-driving cars, surveillance cameras, robotics.

3.4.3 Machine Translation

Map a sentence in one language to a sentence in another.

- Input $\mathcal{X}$: sequence of words in source language (e.g. English).
- Output $\mathcal{Y}$: sequence of words in target language (e.g. Urdu).
- Modern systems (e.g. Google Translate) are powered by large neural networks trained on millions of sentence pairs.

4 Unsupervised Learning

4.1 Definition

In unsupervised learning:

- The dataset contains **no labels** — only inputs $\{x_i\}$.
- The task is to discover **hidden structure** in the data.

Real-Life Analogy: You receive thousands of customer purchase records with no pre-assigned categories. You notice some customers always buy sports gear, others always buy books. Without anyone telling you, you have *discovered* two meaningful groups (clusters).

4.2 Clustering

Clustering is the most common unsupervised learning task:

- Group data points into **clusters** such that points in the same cluster are more similar to each other than to points in other clusters.
- "Similarity" is measured by a distance metric (commonly Euclidean distance).

Classic Applications:

- **Clustering Genes:** group genes with similar expression patterns across different conditions. This helps biologists discover genes that work together (co-expressed), potentially revealing disease pathways.
- **Market Segmentation:** group customers by purchase behaviour to target marketing.
- **Document Clustering:** group news articles by topic without manual labelling.
- **Anomaly Detection:** data points that do not fit any cluster may be anomalies (e.g. fraudulent transactions).

Key Idea

Key difference from supervised learning: In supervised learning, you *tell* the algorithm which group each training point belongs to. In clustering, the algorithm *discovers* the groups itself.

5 Reinforcement Learning

5.1 Definition

Reinforcement Learning (RL) involves an **agent** that:

1. Observes the current **state** of the environment.
2. Takes an **action**.
3. Receives a **reward** (positive) or **penalty** (negative) from the environment.
4. Updates its **policy** (strategy) to maximise cumulative future rewards.

Key Entities:

Term	Meaning
Agent	The learner / decision maker.
Environment	The world the agent interacts with.
State	A description of the current situation.
Action	A decision taken by the agent.
Reward	Feedback signal: positive for good actions, negative for bad.
Policy	The agent's strategy: maps states to actions.

Real-Life Analogy: Training a dog with treats. The dog (agent) performs tricks (actions); you give a treat (positive reward) for sitting, or say "No!" (negative reward) for jumping on guests. Over time the dog learns to sit on command.

5.2 AlphaGo — A Famous Example

What is AlphaGo? AlphaGo is a computer program developed by DeepMind (Google) that plays the ancient board game *Go*.

Why is it significant?

- Go has more possible board positions ($\approx 10^{170}$) than atoms in the observable universe — far too many to search by brute force.
- AlphaGo defeated the world champion Lee Sedol (9-dan) in 2016, widely considered a landmark moment in AI.

Evolution of AlphaGo:

1. **AlphaGo** — learned by studying millions of human games (supervised pre-training) then refined with RL.
2. **AlphaGo Zero** — started with *no human knowledge*; learned entirely from self-play using RL. Stronger than original AlphaGo within days.
3. **MuZero** — does not even need to be told the *rules* of the game; it learns them from scratch through play.

Key Idea

Insight: RL can achieve *superhuman* performance even without human demonstrations, purely by exploring the environment and maximising a reward signal.

6 Real-World ML Applications

Machine Learning has moved from research labs into everyday life.

6.1 Machine Translation

Modern translation systems (Google Translate, DeepL) use deep neural networks trained on billions of sentence pairs. They have dramatically improved in quality and help people communicate across language barriers globally.

6.2 Autonomous Driving

Self-driving cars use ML for:

- Perception: detecting pedestrians, vehicles, road signs (object detection).
- Planning: deciding when to brake, accelerate, or turn (reinforcement learning + other techniques).

Potential impact: reduce the 1.35 million annual road-accident deaths (WHO, 2018) and cut traffic congestion.

6.3 Healthcare

- **Chest X-ray interpretation:** deep learning models can detect pneumonia, tuberculosis, and COVID-19 findings with radiologist-level accuracy.
- **Diabetic retinopathy:** ML models analyse fundoscopy images to detect early signs of blindness-causing retina damage.
- **Drug discovery:** ML models predict how COVID-19 infected cells respond to certain drugs, dramatically accelerating research.

6.4 Large Language Models (LLMs)

Systems such as GPT-4, Gemini, and Claude are trained on vast text corpora using self-supervised learning. They can generate text, answer questions, write code, and summarise documents — revolutionising how humans interact with computers.

7 ML Risks and Ethical Concerns

With great power comes great responsibility. The lecture identifies the following risks:

- **Privacy Concerns** — models trained on personal data may leak sensitive information (e.g. inferring medical conditions from buying patterns).

- **Security Risks** — adversarial attacks can fool ML models (e.g. adding invisible noise to an image causes a self-driving car to misclassify a stop sign).
- **Bias** — if training data reflects historical biases (racial, gender), the model learns and amplifies those biases.
- **Fairness** — an ML-based hiring tool that unfairly disadvantages certain demographic groups.
- **Fake Content** — generative models can produce deepfake videos and misinformation at scale.
- **Energy Consumption** — training large neural networks can emit as much CO_2 as several transatlantic flights.

Key Idea

Responsible AI: As ML practitioners, it is our duty to be aware of these risks, mitigate bias in data collection, protect user privacy, and consider the environmental impact of large-scale training.

8 Linear Regression — The Simplest ML Model

Linear regression is the *foundation* of supervised machine learning. Every advanced model (neural networks, SVMs, etc.) builds upon its principles. It assumes the output y is a *linear* function of the inputs.

8.1 Representing the Hypothesis h

8.1.1 Single Feature

For one input feature x (e.g. house size):

$$h(x) = \theta_0 + \theta_1 x$$

where:

- θ_0 — **bias / intercept**: the predicted value when $x = 0$.
- θ_1 — **weight / slope**: how much h changes per unit increase in x.

Geometric meaning: This is the equation of a straight line. θ_0 shifts the line up/down; θ_1 controls its steepness.

8.1.2 Multiple Features

For n features $x_1, x_2, \ldots, x_n$:

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Using summation notation (and defining $x_0 = 1$):

$$\boxed{h(\mathbf{x}) = \sum_{i=0}^{n} \theta_i \, x_i}$$

Variable definitions:

Symbol	Meaning
x_i	i-th feature value
θ_i	Weight (parameter) for the i-th feature
$x_0 = 1$	Dummy feature enabling θ_0 (bias) to fit in the sum
n	Number of features

8.2 The Three Pillars of Linear Regression

The lecture identifies three core components needed to solve any supervised learning problem:

Component	Role	Example
Hypothesis Class	Family of possible predictors	All straight lines (linear functions)
Loss Function	Measures how bad a prediction is	Squared difference between prediction and truth
Optimisation Algorithm	Finds the best predictor in the hypothesis class	Gradient Descent

8.3 Hypothesis Class

Definition: The hypothesis class is the *family of all functions* the learning algorithm is allowed to consider.

For linear regression we restrict ourselves to linear predictors:

$$\boxed{h(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})}$$

Variable Definitions:

- $\mathbf{w} = [w_0,\ w_1,\ w_2,\ \ldots,\ w_n]^\top$
  Weight vector — the parameters we learn. Each w_i is the importance given to feature i.
- $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x_1,\ x_2,\ \ldots,\ x_n]^\top$
  Feature extractor / feature vector — represents the input. The leading 1 allows the bias term w_0 to act as an intercept.
- $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$
  Dot product (Score) — sums up each weight times its corresponding feature.

Why use vector notation? It generalises cleanly to any number of features, and maps directly to efficient matrix operations in code (NumPy, PyTorch, etc.).

Concrete Example: Suppose $n = 1$ (only house size), $\mathbf{w} = [1,\ 0.57]^\top$, and $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x]^\top$ where $x = 1000$ sq. ft.

$$h(x) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) = 1 \cdot 1 + 0.57 \cdot 1000 = 1 + 570 = 571$$

So the model predicts a price of \$571 (in thousands, presumably).

8.4 Loss Function

8.4.1 Residual

The **residual** for a single training example $(x^{(i)}, y^{(i)})$ is the difference between the prediction and the true value:

$$\text{residual}^{(i)} = h(\mathbf{x}^{(i)}) - y^{(i)}$$

A large residual means a poor prediction.

8.4.2 Squared Loss (per example)

$$\boxed{\text{Loss}(\mathbf{x},\, y,\, \mathbf{w}) = \frac{1}{2}[h(\mathbf{x}) - y]^2 = \frac{1}{2}\left[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) - y\right]^2}$$

Why squared?

- Squaring makes the loss always non-negative.
- Squaring penalises *large* errors much more than small ones.
- The derivative (needed for optimisation) is clean.
- The $\frac{1}{2}$ factor is a mathematical convenience — it cancels the 2 that appears during differentiation.

8.4.3 Training Loss (over all N examples)

To evaluate performance on the whole training set, we take the *average* squared loss:

$$\boxed{\text{TrainLoss}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^{N} \left[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)}\right]^2}$$

Variable Definitions:

- N — number of training examples.
- $\mathbf{x}^{(i)}$ — feature vector of the i-th training example.
- $y^{(i)}$ — true output label of the i-th training example.
- $\mathbf{w}$ — weight vector (what we are optimising over).

Real-Life Analogy: Imagine you are a teacher grading essays. Each residual is how far a student's mark is from the correct answer. The training loss is the *average wrongness* over all students. You want to find the grading rubric ($\mathbf{w}$) that minimises total wrongness.

8.5 Gradient of the Training Loss

To minimise TrainLoss($\mathbf{w}$), we need its gradient with respect to $\mathbf{w}$:

$$\boxed{\nabla_{\mathbf{w}} \operatorname{TrainLoss}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \underbrace{\left[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)}\right]}_{\text{residual}} \boldsymbol{\phi}(\mathbf{x}^{(i)})}$$

How to read this: For each training example:

1. Compute the residual (prediction − truth).
2. Multiply the residual by the feature vector $\boldsymbol{\phi}(\mathbf{x}^{(i)})$.

The gradient is the average of these products over all examples.

Intuition:

- If the gradient is *positive* in some direction, increasing $\mathbf{w}$ in that direction *increases* the loss — so we should *decrease* $\mathbf{w}$ there.
- If the gradient is *negative*, we should *increase* $\mathbf{w}$.
- The gradient always points **uphill**; to minimise, we move **downhill** (opposite to the gradient).

8.6 Optimisation Algorithm: Gradient Descent

Goal: Find $\mathbf{w}^* = \arg\min_{\mathbf{w}} \operatorname{TrainLoss}(\mathbf{w})$.

Gradient Descent Algorithm:

1. Initialise $\mathbf{w}$ (often to all zeros, or small random values).
2. Repeat until convergence:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla_{\mathbf{w}} \operatorname{TrainLoss}(\mathbf{w})$$

Variable Definitions:

- η (*eta*) — **learning rate** (step size). Controls how big a step we take downhill at each iteration.
- $\nabla_{\mathbf{w}} \operatorname{TrainLoss}(\mathbf{w})$ — gradient computed over *all* N training examples.

Real-Life Analogy (Mountain Descent): Imagine you are blindfolded on a hilly landscape and want to reach the lowest point (valley = minimum loss). At each step you:

- Feel the slope of the ground under your feet (= compute gradient).
- Take a step in the direction that goes most steeply downward.

The learning rate η is the size of your step.

Possible Problems with Gradient Descent:

- **Local minima:** In non-convex problems, GD may get stuck in a local minimum rather than the global minimum. (Note: for linear regression the loss is *convex*, so local = global minimum.)
- **Too large η:** The algorithm may overshoot and *diverge* (oscillate or blow up).
- **Too small η:** Convergence is very slow.

8.7 Gradient Descent on Large Datasets

Problem: Standard (Batch) Gradient Descent computes the gradient over *all* N training examples at every update step.

$$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \left[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)}\right] \boldsymbol{\phi}(\mathbf{x}^{(i)})$$

If $N = 10^7$ (10 million examples) this is *very* expensive per update.

8.8 Stochastic Gradient Descent (SGD)

Idea: Instead of summing over all N examples, update $\mathbf{w}$ using *one example at a time* (or a small **mini-batch**).

SGD Update Rule (single example i):

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} - \eta \left[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)}\right] \boldsymbol{\phi}(\mathbf{x}^{(i)})}$$

Mini-batch SGD: Use a small batch B (e.g. $B = 32$ or $B = 256$) per update. Balances speed and stability.

Variant	Examples per update	Speed	Stability
Batch GD	All N	Slow (large N)	Very stable
SGD	1	Very fast	Noisy
Mini-batch SGD	B (e.g. 32)	Fast	Moderate

Real-Life Analogy:

- **Batch GD** $\Rightarrow$ reading every student review before adjusting your teaching style.
- **SGD** $\Rightarrow$ adjusting your style after every single student comment — faster but more erratic.
- **Mini-batch SGD** $\Rightarrow$ reading a handful of reviews, then adjusting — a practical balance.

Key Idea

Why SGD works: Even though each individual gradient estimate is noisy, on average over many steps the noisy gradients point in the right direction. In practice,

SGD with mini-batches is the workhorse of deep learning.

9 Worked Examples

9.1 Example 1 — Computing Training Loss

Example

Setup: Suppose we have three training points and a model $h(x) = \mathbf{w}^\top \boldsymbol{\phi}(x)$ with $\mathbf{w} = [1,\ 0.57]$, $\boldsymbol{\phi}(x) = [1,\ x]$.

i	$x^{(i)}$	$y^{(i)}$
1	500	300
2	1000	600
3	1500	800

Step 1 — Compute predictions:

$$\begin{aligned} h(500) &= 1 + 0.57 \times 500 = 1 + 285 = 286 \\ h(1000) &= 1 + 0.57 \times 1000 = 1 + 570 = 571 \\ h(1500) &= 1 + 0.57 \times 1500 = 1 + 855 = 856 \end{aligned}$$

Step 2 — Compute residuals:

$$\begin{aligned} r^{(1)} &= 286 - 300 = -14 \\ r^{(2)} &= 571 - 600 = -29 \\ r^{(3)} &= 856 - 800 = +56 \end{aligned}$$

Step 3 — Compute squared losses:

$$\begin{aligned} \text{Loss}^{(1)} &= \tfrac{1}{2}(-14)^2 = \tfrac{1}{2}(196) = 98 \\ \text{Loss}^{(2)} &= \tfrac{1}{2}(-29)^2 = \tfrac{1}{2}(841) = 420.5 \\ \text{Loss}^{(3)} &= \tfrac{1}{2}(+56)^2 = \tfrac{1}{2}(3136) = 1568 \end{aligned}$$

Step 4 — Training Loss:

$$\text{TrainLoss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \text{Loss}^{(i)} = \frac{98 + 420.5 + 1568}{3} = \frac{2086.5}{3} \approx 695.5$$

Interpretation: On average, the model's predictions are off by $\sqrt{2 \times 695.5} \approx 37.3$ units (this approximation follows from the $\frac{1}{2}$ factor in the loss definition).

Alternative Method / Trick

Shortcut — Root Mean Squared Error (RMSE): A commonly reported al-

ternative metric is:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(h(x^{(i)}) - y^{(i)})^2}$$

RMSE is in the *same units* as y (e.g. dollars for house prices), making it easier to interpret.

9.2 Example 2 — One Step of Gradient Descent

Example

Using the same setup as Example 1, perform one step of gradient descent with learning rate $\eta = 0.00001$.

Step 1 — Compute gradient:

$$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} r^{(i)}\, \boldsymbol{\phi}(\mathbf{x}^{(i)})$$

For $\mathbf{w} = [w_0,\ w_1]$:

$$\frac{\partial}{\partial w_0}\text{TrainLoss} = \frac{1}{3}[(-14)(1) + (-29)(1) + (+56)(1)] = \frac{-14 - 29 + 56}{3} = \frac{13}{3} \approx 4.33$$

$$\begin{aligned}\frac{\partial}{\partial w_1}\text{TrainLoss} &= \frac{1}{3}[(-14)(500) + (-29)(1000) + (+56)(1500)] \\ &= \frac{-7000 - 29000 + 84000}{3} = \frac{48000}{3} = 16000\end{aligned}$$

Step 2 — Update weights:

$$\begin{aligned}w_0 &\leftarrow 1 - 0.00001 \times 4.33 = 1 - 0.0000433 \approx 0.99996 \\ w_1 &\leftarrow 0.57 - 0.00001 \times 16000 = 0.57 - 0.16 = 0.41\end{aligned}$$

Interpretation: After one step, w_1 decreased from 0.57 to 0.41, meaning the model reduced the slope — intuitive because the large positive residual on example 3 was pulling the loss up.

10 Practice Questions

Practice Questions

Topic: ML Taxonomy

Q1 (Easy): A hospital collects patient records (age, blood pressure, cholesterol level) and labels each record as "heart disease present" or "not present". Which branch of ML is this?

Answer: Supervised learning — classification problem. $\mathcal{X}$ = patient features, $\mathcal{Y} \in$

$\{0, 1\}$.

Q2 (Medium): An e-commerce company wants to group its millions of products into categories automatically, without any human-assigned labels. Which ML approach should be used and why?
Answer: Unsupervised learning, specifically **clustering** (e.g. K-means). There are no labels, so supervised learning cannot be applied directly. The algorithm groups products by similarity of their descriptions, images, or purchase co-occurrence.

Q3 (Exam-level): Define the **hypothesis function** for supervised learning and explain the difference between regression and classification. Give one real-world example of each and write their formal notation.
Answer: A hypothesis function $h : \mathcal{X} \to \mathcal{Y}$ maps input features to predicted outputs learned from training data.

- **Regression:** $\mathcal{Y} \subseteq \mathbb{R}$ (continuous output).
  Example: predicting electricity consumption given weather data.
  $h : \mathbb{R}^n \to \mathbb{R}$
- **Classification:** $\mathcal{Y} = \{c_1, c_2, \ldots, c_k\}$ (discrete classes).
  Example: classifying handwritten digits 0–9.
  $h : \mathbb{R}^n \to \{0, 1, 2, \ldots, 9\}$

Topic: Linear Regression & Gradient Descent

Q4 (Easy): What is the squared loss for a single prediction where $h(x) = 7$ and the true label $y = 10$?

$$\text{Loss} = \tfrac{1}{2}(7 - 10)^2 = \tfrac{1}{2}(9) = 4.5$$

Q5 (Medium): You have a linear model with $\mathbf{w} = [2,\ 3]^\top$ and feature vector $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 4]^\top$. True label $y = 20$.
(a) What is the prediction $h(\mathbf{x})$?
(b) What is the residual?
(c) What is the contribution of this example to $\nabla_{\mathbf{w}}\text{TrainLoss}$?
Solution:
(a) $h = \mathbf{w}^\top \boldsymbol{\phi} = 2(1) + 3(4) = 2 + 12 = 14$
(b) residual $= 14 - 20 = -6$
(c) Gradient contribution $= r \cdot \boldsymbol{\phi} = -6 \times [1,\ 4]^\top = [-6,\ -24]^\top$
So the gradient update would *increase* $\mathbf{w}$ (because we subtract a negative gradient), which increases the prediction to reduce the residual.

Q6 (Exam-level): Explain the difference between Batch Gradient Descent, SGD, and Mini-batch SGD. What are the trade-offs of each? When would you use each in practice?
Answer:

Method	Trade-offs	When to use
Batch GD	Exact gradient; computationally expensive for large N. Stable convergence.	Small datasets that fit in memory ($N < 10^4$).
SGD	Very fast updates; noisy gradient; may not converge smoothly. Can escape local minima due to noise.	Online learning; extremely large datasets; real-time learning.
Mini-batch SGD	Balance of speed and stability. Efficient on GPU due to vectorisation. Most widely used in practice.	Neural network training; most modern ML workloads.

11 Key Takeaways

Quick Summary

- **ML** uses data to learn model parameters for future predictions.
- Three paradigms: **Supervised**, **Unsupervised**, **Reinforcement** learning.
- **Supervised learning**: learn from labelled (x, y) pairs; regression $\Rightarrow$ continuous y; classification $\Rightarrow$ discrete y.
- **Unsupervised learning**: no labels; goal is to discover structure (clustering, density estimation).
- **Reinforcement learning**: agent maximises reward through trial and error in an environment.
- **Linear regression** hypothesis: $h(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$.
- **Squared loss** per example: $\frac{1}{2}[\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) - y]^2$.
- **Training loss**: average squared loss over all N examples.
- **Gradient Descent**: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$.
- **SGD / Mini-batch SGD**: practical alternatives to batch GD; update on one example or a small batch at a time.
- **ML risks**: bias, fairness, privacy, security, energy — responsible ML is essential.